ROS2 Based Tasks – USE ROS2 HUMBLE

Task 1: Simulating a robot by creating a world frame and moving init.

- Create a 2-wheel differential drive robot using URDF.
- Attach the following sensors:
  - LIDAR (mounted on top of the robot)
  - IMU
  - Wheel odometry
- Publish LIDAR and IMU data on their respective topics.
- Visualize the sensor data using RViz.
- Create a custom world in Gazebo and spawn the robot into this world.
- Create a map and integrate Nav2:
  - The robot should be able to navigate to a given goal.
  - Implement obstacle avoidance during navigation.

Task 2: Odometry Publisher from Simulated Wheel Encoder Ticks

Objective:

Simulate realistic robot behavior by implementing differential drive odometry computation using encoder tick values — similar to real hardware scenarios.

Background:

You are provided with a ROS2 Python node that simulates a differential drive robot by publishing **cumulative wheel encoder ticks** on the following topics:

- `/left_wheel_ticks` (`std_msgs/Int32`)
- `/right_wheel_ticks` (`std_msgs/Int32`)

These ticks increase as if the robot is moving forward. In actual robots, these ticks come from encoders attached to the wheels. The purpose of this task is to help you simulate and understand the motion model of a robot using basic odometry.

Instructions:

- **Create a ROS2 package** (Python/C++) named `odom_computation`.
- Write a Python node:
- **Subscribes to**:
  - `/left_wheel_ticks`
  - `/right_wheel_ticks`
- **Publishes to**:
  - **`/odom`** using `nav_msgs/msg/Odometry`
- **Calculates** the robot's pose (`x`, `y`, `theta`) and velocity using differential drive logic.
- Use the following **robot parameters**:
  - `wheel_radius = 0.065` meters
  - `wheel_base = 0.21` meters
  - `ticks_per_revolution = 512`
- Convert heading (`theta`) into quaternion for the `orientation` field in the Odometry message.
- Update the Odometry message at **10 Hz**.

Expected Behavior:

- The **`/odom`** topic should begin publishing once the node starts running.
- The published odometry message should reflect **forward motion** (i.e., increasing `x` over time, `y ≈ 0`, and constant `theta ≈ 0`).
- Orientation must be published in **quaternion format** (converted from heading `theta`).
- Both `pose` and `twist` fields in the message should contain valid data.
- **To verify correctness**, use the following command:

```
ros2 topic echo /odom --once
```

or:

```
ros2 topic echo /odom
```

to monitor continuous updates.

Optional (Bonus Task):

- Implement an additional ROS2 node named `imu_publisher.py` that simulates basic IMU data and publishes to `/imu/data` using `sensor_msgs/msg/Imu`.
  - Populate fields like linear acceleration and angular velocity with realistic or synthetic values.
  - Use zero/noise-free data or simulated movement patterns.
  - This can help test IMU integration for future localization pipelines.

References:

- **URDF Robot Creation**: https://docs.ros.org/en/humble/Tutorials/Intermediate/URDF/Using-URDF-With-Robot-State-Publisher.html
- **Add LIDAR and IMU to URDF**: https://classic.gazebosim.org/tutorials?tut=ros_imu https://automaticaddison.com/how-to-launch-a-lidar-sensor-in-ros-2-foxy/ *(still applicable for Humble)*
- **Creating Custom Gazebo Worlds**: https://classic.gazebosim.org/tutorials?tut=build_world
- **Nav2 Setup and Integration**: https://navigation.ros.org/ https://docs.ros.org/en/humble/Tutorials/Navigation2/Getting-Started.html
- Odometry Message Structure nav_msgs/Odometry documentation
- IMU Message Format sensor_msgs/Imu documentation
- Quaternion Conversion tf_transformations in Python
- ROS2 Python Publisher/Subscriber Example Writing a Simple ROS2 Python Publisher and Subscriber